Labs 2 and 3

Run this Lab Example:

Step 1 — Working on the ECG Arrhythmia Example in Lecture 2 and 3 notes

UCI Arrhythmia: demographic and ECG-derived measurements from 452 patient records. The dataset is designed to distinguish normal ECG records from different types of cardiac arrhythmia. It contains 279 input features and one diagnosis class column. Arrhythmia - UCI Machine Learning Repository

NOTE: Notice how the link above is pasted as "Arrhythmia - UCI Machine Learning Repository" not "https://archive.ics.uci.edu/dataset/5/arrhythmia". While seeking approval for Phase 1 assessment, paste the link as shttps://archive.ics.uci.edu/dataset/5/arrhythmia in the MS Form.

Check	Result
Tabular, alpha-numeric	Yes — delimited tabular data containing categorical, integer, and real-valued demographic and ECG measurements
Feature count	279 input features, plus 1 diagnosis class column — 280 columns in the raw file
Enough records for a first look?	Yes — 452 patient records, suitable for data inspection, cleaning, visualisation, and an introductory classification example

Step 2 — Document it

Field	Answer
Dataset name	UCI Arrhythmia Dataset (arrhythmia.data)
Source	https://archive.ics.uci.edu/dataset/5/arrhythmia
Direct file	https://archive.ics.uci.edu/static/public/5/arrhythmia.zip
Access date	21 July 2026
Licence	Creative Commons Attribution 4.0 International — CC BY 4.0. The dataset may be shared and adapted provided appropriate credit is given.
Original collection	Patient demographic and ECG-derived measurements collected to distinguish normal ECGs from cardiac arrhythmias and classify records into 16 diagnosis groups. Cardiologist classifications were treated as the reference standard, and patient names and identification numbers were removed.

How the data was actually collected, in plain terms: Each row represents one patient whose demographic details and ECG measurements were recorded. The dataset includes age, sex, height, weight, heart rate, ECG timing intervals, wave shapes, amplitudes, and measurements taken across 12 ECG leads. A cardiologist assigned each record to the normal group or an arrhythmia category, and this clinical classification was treated as the reference result for developing machine-learning methods. Patient names and identification numbers were removed. Some ECG

measurements are missing, although the documentation does not explain the specific reason each value was unavailable.

Step 3 — Load and label the Arrhythmia dataset

Load the same UCI Arrhythmia file used in the Week 2 and Week 3 notes. The raw file has no header row and uses a question mark to represent a missing value.

```
import pandas as pd

df = pd.read_csv(
    "arrhythmia.csv",
    header=None,
    na  values="?"
)

print("Raw shape:", df.shape)
print(df.head())
```

Assign meaningful names to the first 15 columns and the diagnosis column. The remaining ECG waveform columns keep their numeric labels.

```
df.columns = [
    "age", "sex", "height", "weight",
    "qrs_duration", "pr_interval", "qt_interval",
    "t_interval", "p_interval", "qrs_angle",
    "t_angle", "p_angle", "qrst_angle",
    "j_angle", "heart_rate"
] + list(df.columns[15:-1]) + ["class"]

print(df.columns[:15])
print(df.columns[-3:])
print(df.head())
```

1. The expected raw shape is 452 rows and 280 columns.
2. Each row represents one patient ECG record.
3. Class 1 means normal; classes 2–15 are named arrhythmia types; class 16 is unclassified.
4. Do not continue until the file loads correctly and the column count remains 280.

Task: Run both code blocks. Record the raw shape and explain what one row and the final class column represent.

Step 4 — Describe the structure and protect the raw data

Create an untouched copy before inspecting or cleaning. All later transformations must be applied to a copy, not directly to df or df_original.

```
df_original = df.copy(deep=True)

print("Records:", df_original.shape[0])
print("Columns:", df_original.shape[1])
print("\nData-type counts:")
print(df_original.dtypes.value_counts())

df_original.info()
```

```
print("\nSummary of numeric columns:")
print(df_original.describe())
```

5. shape reports the number of patient records and columns.
6. dtypes.value_counts() shows how many columns use each data type.
7. info() reports non-missing values and memory use.
8. describe() provides numerical summaries, but the output must still be interpreted critically.
9. df_original must remain unchanged throughout the lab.

Task: Identify the demographic variables, ECG variables and target column. Note any data types that may need conversion or special treatment.

Step 5 — Inspect data quality before deciding what to clean

Do not remove values only because they appear unusual. First inspect missingness, duplicate rows, ranges and related variables.

```
# Missing values
missing = df_original.isna().sum()
print("Columns containing missing values:")
print(missing[missing > 0].sort_values(ascending=False))

# Duplicate rows
print("\nNumber of duplicate rows:")
print(df_original.duplicated().sum())

# Important ranges
range_cols = ["age", "height", "weight", "heart_rate"]
print("\nMinimum and maximum values:")
print(df_original[range_cols].agg(["min", "max"]))

# Diagnosis distribution
print("\nPatients per diagnosis class:")
print(df_original["class"].value_counts().sort_index())
```

Review potentially unusual demographic records by considering age, height and weight together.

```
review_cols = ["age", "sex", "height", "weight", "class"]

records_to_review = df_original.loc[
    (df_original["age"] < 18)
    | (df_original["height"] < 120)
    | (df_original["height"] > 220)
    | (df_original["weight"] < 25),
    review_cols
].sort_values(["age", "height", "weight"])

records_to_review = records_to_review.copy()
records_to_review["bmi"] = (
    records_to_review["weight"]
    / (records_to_review["height"] / 100) ** 2
)

print(records_to_review.round(2).to_string())
```

10. Heights of 608 cm and 780 cm are physically impossible.
11. A low age or weight is not automatically invalid because the dataset is not documented as adult-only.
12. BMI is used only as a consistency check; adult BMI categories must not be applied automatically to children.

13. A cleaning decision must be supported by documentation or by an internally inconsistent combination of values.

Task: Write two short notes: one value you can confidently treat as invalid, and one unusual value that should not be removed automatically.

Step 6 — Define, apply and verify the cleaning function

The cleaning function places all agreed rules in one reusable block. It works on a deep copy so the supplied DataFrame remains unchanged.

```
def clean_data(data):
    clean = data.copy(deep=True)
    clean["sex"] = clean["sex"].replace(
        {"M": 0, "F": 1, "m": 0, "f": 1}
    )
    clean["bmi"] = (
        clean["weight"] /
        (clean["height"] / 100) ** 2
    )
    invalid_record = (
        (clean["height"] > 250)
        | ((clean["age"] <= 1) & (clean["height"] > 100))
        | ((clean["age"] >= 20) & (clean["bmi"] < 10))
    )
    print("Clearly unreliable records removed:")
    print(
        clean.loc[
            invalid_record,
            ["age", "sex", "height", "weight", "bmi", "class"]
        ].round(2)
    )
    clean = clean.loc[~invalid  record].copy()
    clean["bmi"] = (
        clean["weight"] /
        (clean["height"] / 100) ** 2
    )
    for col in ["p_angle", "t_angle", "qrst_angle", "heart_rate"]:
        clean[col] = clean[col].fillna(clean[col].median())
    clean = clean.drop(columns=["j_angle"], errors="ignore")
    clean = clean.drop_duplicates()
    clean["arrhythmia_present"] = clean["class"] != 1
    clean = clean.reset_index(drop=True)
    return clean
```

Apply the function and inspect the result before storing or analysing it.

```
df_etl_clean = clean_data(df_original)

print("Original shape:", df_original.shape)
print("Cleaned shape:", df_etl_clean.shape)
print("Rows removed:", len(df_original) - len(df_etl_clean))

print("\nCleaned demographic ranges:")
print(
    df_etl_clean[
        ["age", "height", "weight", "bmi"]
    ].agg(["min", "max"])
```

```
)

print("Heights above 250 cm:",
      (df_etl_clean["height"] > 250).sum())
print("Adults with BMI below 10:",
      ((df_etl_clean["age"] >= 20)
       & (df_etl_clean["bmi"] < 10)).sum())
print("Duplicate rows:",
      df_etl_clean.duplicated().sum())
print("j_angle still present:",
      "j_angle" in df_etl_clean.columns)
```

14. The expected cleaned shape under the current rules is 448 rows and 281 columns.
15. The column count becomes 281 because j_angle is dropped while bmi and arrhythmia_present are added.
16. The original df_original should still contain all 452 raw records.
17. The verification output should show that the intended invalid combinations are no longer present.

Task: Confirm the number of removed rows and columns. Explain why the cleaned table has one more column than the raw table.

Step 7 — Demonstrate ETL and ELT, then complete Part A

SQLite is used only to demonstrate the Load stage. It stores database tables in a single local file and does not require a separate server.

```
import sqlite3

# ETL: transform first, then load

with sqlite3.connect("etl.db") as conn:
    df_etl_clean.to_sql(
        "clean",
        conn,
        if_exists="replace",
        index=False
    )

# ELT: load raw first, then transform
with sqlite3.connect("elt.db") as conn:
    df_original.to_sql(
        "raw",
        conn,
        if_exists="replace",
        index=False
    )

    df_elt_raw = pd.read_sql(
        "SELECT * FROM raw",
        conn
    )

    df_elt_clean = clean_data(df_elt_raw)

    df_elt_clean.to_sql(
        "clean",
        conn,
        if_exists="replace",
```

```
        index=False
    )

print("ETL clean shape:", df_etl_clean.shape)
print("ELT raw shape:", df_elt_raw.shape)
print("ELT clean shape:", df_elt_clean.shape)
```

Confirm which tables were stored.

```
with sqlite3.connect("etl.db") as conn:
    print("Tables in etl.db:")
    print(pd.read_sql(
        "SELECT name FROM sqlite_master "
        "WHERE type='table'",
        conn
    ))

with sqlite3.connect("elt.db") as conn:
    print("\nTables in elt.db:")
    print(pd.read_sql(
        "SELECT name FROM sqlite_master "
        "WHERE type='table'",
        conn
    ))
```

Complete the remaining Part A demonstrations from the notes using the Arrhythmia dataset:

18. Compare a Pandas Series with a Pandas DataFrame.
19. Run the loop-versus-vectorised weight/height calculation and compare execution time.
20. Recheck shape, data-type counts, info(), memory use and describe().
21. Explain why code that runs without an error can still produce a meaningless summary.

```
# Series versus DataFrame
print(df_original["heart_rate"].mean())
print(
    df_original[
        ["heart_rate", "age"]
    ].mean()
)

import time
# Slow method: loop
start = time.time()
slow_result = []
for i in range(len(df_original)):
    slow_result.append(
        df_original["weight"].iloc[i] / df_original["height"].iloc[i]
    )
slow_time = time.time() - start
# Fast method: vectorised calculation
start = time.time()
fast_result = df_original["weight"] / df_original["height"]
fast_time = time.time() - start
# Compare the first five results
print("Slow result:")
print(slow_result[:5])
print("\nFast result:")
print(fast_result.head().tolist())
```

```
# Compare the speed
print("\nSlow time:", slow_time)
print("Fast time:", fast_time)
print("\nFast method is", slow  time / fast  time, "times faster")

# Shape and data types
print(df_original.shape)
print(df  original.dtypes.value  counts())
df_original.info()
```

Task: State the difference between ETL and ELT. Verify that etl.db contains only clean, while elt.db contains raw and clean.

Step 8 — Complete every Part B exploration activity

Use df_etl_clean for cleaned-data summaries and visualisations. Use df_original only when the notes specifically ask you to inspect the raw data or compare raw and cleaned results. Run every code block, retain the output, and add your own short interpretation beneath each result.

Important: The figures printed in the notes are examples. You must execute the code yourself and interpret the output generated by your own notebook.

8.1 Descriptive statistics and target distribution

```
summary_cols = ["age", "height", "weight", "heart_rate"]

print(df_etl_clean[summary_cols].describe())

skew_results = (
    df_etl_clean[summary_cols]
    .skew()
    .round(2)
)
print(skew_results)

print(
    df_etl_clean["arrhythmia_present"]
    .value_counts(normalize=True)
)

print("Minimum age retained:", df_etl_clean["age"].min())
print("Minimum weight retained:", df_etl_clean["weight"].min())
```

22. Describe the centre and spread of age, height, weight and heart rate.
23. State which variables are left-skewed, right-skewed or approximately symmetric.
24. Report the proportions with and without arrhythmia.

Task: Write three or four sentences explaining the most important descriptive results.

8.2 Histograms before and after cleaning

```
import matplotlib.pyplot as plt

# Cleaned-data distributions
plt.hist(df_etl_clean["heart_rate"].dropna(), bins=20)
plt.xlabel("Heart rate (bpm)")
plt.ylabel("Number of patients")
plt.title("Distribution of heart rate")
plt.show()
```

```
plt.hist(df_etl_clean["age"].dropna(), bins=25)
plt.xlabel("Age")
plt.ylabel("Number of patients")
plt.title("Distribution of age")
plt.show()

# Raw distributions
for col, label, unit in [
    ("height", "Height", "cm"),
    ("weight", "Weight", "kg"),
    ("heart_rate", "Heart rate", "bpm"),
]:
    plt.hist(df  original[col].dropna(), bins=30)
    plt.title(f"{label} before cleaning")
    plt.xlabel(f"{label} ({unit})")
    plt.ylabel("Number of patients")
    plt.show()

# Compare raw and cleaned ranges
for col in ["age", "height", "weight", "heart_rate"]:
    print(
        col,
        "raw range:",
        (df_original[col].min(), df_original[col].max()),
        "clean range:",
        (df_etl_clean[col].min(), df_etl_clean[col].max()),
    )

# Replot the cleaned measurements
for col, label, unit in [
    ("height", "Height", "cm"),
    ("weight", "Weight", "kg"),
    ("heart  rate", "Heart rate", "bpm"),
]:
    plt.hist(df_etl_clean[col].dropna(), bins=30)
    plt.title(f"{label} after cleaning")
    plt.xlabel(f"{label} ({unit})")
    plt.ylabel("Number of patients")
    plt.show()
```

25. Identify the peak, tail and any isolated bars in each histogram.
26. Explain why the raw height distribution is visually compressed by the impossible values.
27. Confirm that cleaning changed only the values targeted by the cleaning rules.

Task: Compare the raw and cleaned height, weight and heart-rate plots. State what changed and what remained similar.

8.3 Boxplots and statistical-outlier detection

```
# Weight boxplot
plt.boxplot(
    df_etl_clean["weight"].dropna(),
    vert=False,
    tick  labels=["Weight"]
)
plt.xlabel("Weight (kg)")
plt.show()
```

```
# IQR flags for demographic variables
def iqr_flag(series):
    q1, q3 = series.quantile([0.25, 0.75])
    iqr = q3 - q1
    return (
        (series < q1 - 1.5 * iqr)
        | (series > q3 + 1.5 * iqr)
    )

age_flag = iqr_flag(df_etl_clean["age"])
height_flag = iqr_flag(df_etl_clean["height"])
weight_flag = iqr_flag(df_etl_clean["weight"])

demographic_flags = age_flag | height_flag | weight_flag

print(
    df_etl_clean.loc[
        demographic_flags,
        ["age", "sex", "height", "weight", "class"]
    ]
    .sort_values(["age", "height", "weight"])
    .to_string()
)

# Heart rate checked separately with a z-score
mean_hr = df_etl_clean["heart_rate"].mean()
std_hr = df_etl_clean["heart_rate"].std()
z_hr = (
    df_etl_clean["heart_rate"] - mean_hr
) / std_hr

print("Heart-rate values with |z| > 3:")
print(
    df_etl_clean.loc[
        z_hr.abs() > 3,
        ["age", "sex", "heart_rate", "class"]
    ]
)
```

28. Explain the box, median, whiskers and points beyond the whiskers.
29. Review the flagged demographic records using age, height and weight together.
30. Treat IQR and z-score results as statistical flags, not automatic deletion rules.

Task: Identify at least two flagged but plausible records and explain why they should be retained.

31. Plot height, heart rate, and weight in separate panels, before any capping — this is the uncapped baseline you will compare against the capped version in 8.4.

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].boxplot(df_etl_clean["height"].dropna(), tick_labels=["Height"])
axes[0].set_title("Height"); axes[0].set_ylabel("cm")
axes[1].boxplot(df_etl_clean["heart_rate"].dropna(), tick_labels=["Heart rate"])
axes[1].set_title("Heart rate"); axes[1].set_ylabel("bpm")
axes[2].boxplot(df_etl_clean["weight"].dropna(), tick_labels=["Weight"])
axes[2].set_title("Weight"); axes[2].set_ylabel("kg")
plt.suptitle("Cleaned data before optional capping")
plt.show()
```

8.4 Optional sensitivity-only capping

```
df_sensitivity = df_etl_clean.copy(deep=True)

df_sensitivity["height_capped"] = (
    df_sensitivity["height"].clip(
        lower=df  sensitivity["height"].quantile(0.01)
    )
)

df_sensitivity["heart_rate_capped"] = (
    df_sensitivity["heart_rate"].clip(
        lower=df_sensitivity["heart_rate"].quantile(0.01),
        upper=df_sensitivity["heart_rate"].quantile(0.99)
    )
)

df_sensitivity["weight_capped"] = (
    df_sensitivity["weight"].clip(
        upper=df_sensitivity["weight"].quantile(0.99)
    )
)

fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].boxplot(
    df_sensitivity["height_capped"].dropna(),
    tick_labels=["Height"]
)
axes[0].set_title("Height (sensitivity-capped)")
axes[0].set_ylabel("cm")

axes[1].boxplot(
    df_sensitivity["heart_rate_capped"].dropna(),
    tick_labels=["Heart rate"]
)
axes[1].set_title("Heart rate (capped)")
axes[1].set  ylabel("bpm")

axes[2].boxplot(
    df_sensitivity["weight_capped"].dropna(),
    tick_labels=["Weight"]
)
axes[2].set_title("Weight (capped)")
axes[2].set_ylabel("kg")

plt.suptitle("Optional sensitivity-only capped view")
plt.show()
```

32. Confirm that df_etl_clean was not modified.
33. Compare the capped plots with the uncapped boxplots.
34. Explain that sensitivity analysis tests whether conclusions depend strongly on extreme values.

Task: State whether capping changes the visual interpretation and why the capped copy is not used as the main dataset.

8.5 Group comparison and correlation heatmap

```
import numpy as np

# Arrhythmia rate by sex
rate = (
    df  etl  clean
    .groupby("sex", observed=False)["arrhythmia_present"]
    .mean()
)
print(rate)

sex_labels = {0: "Male", 1: "Female"}
labels = [sex_labels[int(value)] for value in rate.index]

plt.bar(labels, rate.values)
plt.xlabel("Sex")
plt.ylabel("Proportion with arrhythmia")
plt.title("Arrhythmia rate by sex")
plt.show()

# Correlation heatmap
num_cols = [
    "age", "height", "weight", "qrs_duration",
    "pr_interval", "qt_interval", "heart_rate"
]

corr_matrix = df_etl_clean[num_cols].corr()
print(corr_matrix.round(2))

upper_triangle = corr_matrix.where(
    np.triu(
        np.ones(corr_matrix.shape),
        k=1
    ).astype(bool)
)
strongest  pair = upper  triangle.abs().stack().idxmax()

print(
    "Strongest absolute correlation:",
    strongest_pair,
    round(
        corr_matrix.loc[
            strongest_pair[0],
            strongest_pair[1]
        ],
        2
    )
)

plt.imshow(
    corr  matrix,
    cmap="RdYlGn",
    vmin=-1,
    vmax=1
)
plt.xticks(
```

```
    range(len(num_cols)),
    num_cols,
    rotation=45,
    ha="right"
)
plt.yticks(range(len(num_cols)), num_cols)
plt.colorbar(label="Correlation")
plt.title("Correlation between 7 of 279 input features")
plt.show()
```

35. Explain which sex category has the higher observed arrhythmia proportion.
36. Identify the strongest positive and strongest negative linear relationships.
37. Remember that correlation describes association and does not prove causation.

Task: Write one paragraph interpreting the bar chart and one paragraph interpreting the correlation heatmap.

8.6 Arrhythmia-type profile and missingness

```
class_names = {
    1: "Normal",
    2: "Ischemic changes (CAD)",
    3: "Old Ant. MI",
    4: "Old Inf. MI",
    5: "Sinus tachycardia",
    6: "Sinus bradycardia",
    7: "PVC",
    8: "Supraventricular PC",
    9: "Left bundle branch block",
    10: "Right bundle branch block",
    11: "1st degree AV block",
    12: "2nd degree AV block",
    13: "3rd degree AV block",
    14: "Left vent. hypertrophy",
    15: "Atrial fib./flutter",
    16: "Other",
}

pqrst_cols = [
    "p_interval", "qrs_duration", "pr_interval",
    "qt  interval", "t  interval", "heart  rate"
]

class_counts = (
    df_etl_clean["class"]
    .value  counts()
    .sort_index()
)
print("Patients per class:")
print(class_counts)

profile = (
    df_etl_clean
    .groupby("class")[pqrst_cols]
    .mean()
)
print("Mean profile per class:")
print(profile.round(1))
```

```
profile_z = (
    profile - profile.mean()
) / profile.std()

row_labels = [
    class_names[int(c)]
    for c in profile_z.index
]

plt.imshow(
    profile_z.values,
    cmap="RdYlGn",
    vmin=-2,
    vmax=2,
    aspect="auto"
)
plt.xticks(
    range(len(pqrst  cols)),
    pqrst_cols,
    rotation=45,
    ha="right"
)
plt.yticks(range(len(profile  z)), row  labels)
plt.colorbar(label="Standardised mean (per column)")
plt.title("PQRST + heart rate profile, by arrhythmia type")
plt.show()

# Missingness in the raw data
miss = (
    df_original
    .isna()
    .mean()
    .sort  values(ascending=False)
)
miss = miss[miss > 0] * 100

plt.barh(miss.index.astype(str), miss.values)
plt.xlabel("% missing")
plt.title("Missing values in the raw data")
plt.show()
```

38. Use the class counts before interpreting a class mean; very small classes can produce unstable patterns.
39. Explain what green, yellow and red mean within each standardised feature column.
40. Identify which raw column has the greatest percentage of missing values.

Task: Describe three notable arrhythmia-type patterns and one limitation of interpreting the heatmap.

8.7 Scatter plots and pair plot

```
# Heart rate by arrhythmia type
order = sorted(df_etl_clean["class"].unique())
x_pos = df_etl_clean["class"].map(
    {c: i for i, c in enumerate(order)}
)

plt.scatter(
    x_pos,
    df_etl_clean["heart_rate"],
```

```
    alpha=0.4
)

means = (
    df_etl_clean
    .groupby("class")["heart_rate"]
    .mean()
)
print("Mean heart rate by class:")
print(means.sort_values())

plt.scatter(
    range(len(order)),
    [means[c] for c in order],
    color="red",
    marker="D",
    label="Mean per type"
)
plt.xticks(
    range(len(order)),
    [class_names[c] for c in order],
    rotation=45,
    ha="right"
)
plt.xlabel("Arrhythmia type")
plt.ylabel("Heart rate (bpm)")
plt.title("Heart rate by arrhythmia type")
plt.legend()
plt.show()

# Height versus weight by diagnosis
colors = df_etl_clean["arrhythmia_present"].map(
    {True: "orange", False: "teal"}
)

plt.scatter(
    df_etl_clean["height"],
    df_etl_clean["weight"],
    c=colors,
    alpha=0.5
)
plt.xlabel("Height (cm)")
plt.ylabel("Weight (kg)")
plt.title("Height vs weight, by diagnosis")
plt.show()

# Age versus binary arrhythmia outcome with jitter
jitter = (
    df_etl_clean["arrhythmia_present"].astype(int)
    + np.random.uniform(
        -0.08,
        0.08,
        len(df  etl  clean)
    )
)
```

```
plt.scatter(
    df_etl_clean["age"],
    jitter,
    c=colors,
    alpha=0.5
)
plt.yticks([0, 1], ["No", "Yes"])
plt.xlabel("Age")
plt.ylabel("arrhythmia_present")
plt.title("Age vs arrhythmia_present")
plt.show()

# Pair plot constructed with Matplotlib
cols = [
    "height", "weight",
    "qt_interval", "heart_rate"
]
fig, axes = plt.subplots(4, 4, figsize=(9, 9))

for i, c1 in enumerate(cols):
    for j, c2 in enumerate(cols):
        ax = axes[i, j]
        if i == j:
            ax.hist(
                df_etl_clean[c1].dropna(),
                bins=20
            )
        else:
            ax.scatter(
                df_etl_clean[c2],
                df_etl_clean[c1],
                s=4,
                alpha=0.35,
                c=colors
            )
        if i == 3:
            ax.set_xlabel(c2)
        if j == 0:
            ax.set_ylabel(c1)

plt.tight_layout()

from matplotlib.patches import Patch
fig.legend(handles=[
    Patch(color="teal", label="No arrhythmia"),
    Patch(color="orange", label="Arrhythmia present"),
], loc="upper right")
plt.show()
```

41. Use the printed mean table to identify the highest and lowest heart-rate class averages.
42. Explain the heavy overlap between diagnosis groups in the height-weight and age plots.
43. Identify the positive height-weight relationship and negative QT-heart-rate relationship in the pair plot.

Task: Write two or three sentences beneath each scatter plot and a short overall conclusion beneath the pair plot.

8.8 Population and sample

44. Population: all patients about whom the research question seeks to draw conclusions.

45. Sample: the patient records contained in this dataset after applying the documented cleaning rules.
46. Parameter: the unknown true value in the wider population.
47. Statistic: a value calculated from the sample and used to estimate a population parameter.

```
print(
    "Sample mean age:",
    df_etl_clean["age"].mean()
)
```

Task: Explain why the sample mean is an estimate rather than the true population mean.

Step 9 — Complete every Part C statistical-testing activity

Part C moves from describing this sample to testing whether selected sample patterns provide evidence about a wider population. Use the cleaned ETL DataFrame and interpret the p-values printed by your own run.

48. H0 is the null hypothesis, usually stating no difference or no association.
49. H1 is the alternative hypothesis, stating a difference or association.
50. Use alpha = 0.05 unless another threshold is specified.
51. When $p < 0.05$, reject H0. Otherwise, fail to reject H0.
52. Failing to reject H0 does not prove that H0 is true.

9.1 Independent-samples t-test: heart rate by diagnosis

```
%pip install scipy
from scipy import stats

a = df_etl_clean.loc[
    df_etl_clean["arrhythmia_present"],
    "heart_rate"
]

b = df_etl_clean.loc[
    ~df_etl_clean["arrhythmia_present"],
    "heart_rate"
]

t, p = stats.ttest_ind(
    a,
    b,
    equal  var=False
)

print("Mean with arrhythmia:", a.mean())
print("Mean without arrhythmia:", b.mean())
print(f"t = {t:.2f}, p = {p:.4f}")

if p < 0.05:
    print(
        "Reject H0: the mean heart rates "
        "differ significantly."
    )
else:
    print(
        "Fail to reject H0: no significant "
        "mean difference was detected."
    )
```

53. The t-value measures the mean difference relative to the uncertainty and variation in the two groups.
54. The sign of t shows which group has the higher sample mean; the absolute size reflects the strength of the difference relative to its standard error.
55. The p-value indicates how unusual the result would be if the population means were truly equal.

Task: Report both group means, t, p and the decision about H0. Explain the conclusion in plain language without claiming the means are identical.

9.2 Chi-square test: sex and arrhythmia status

```
table = pd.crosstab(
    df_etl_clean["sex"],
    df_etl_clean["arrhythmia_present"]
)
print(table)

chi2, p, dof, expected = (
    stats.chi2_contingency(table)
)

print(f"chi2 = {chi2:.2f}, p = {p:.6f}")
print("Degrees of freedom:", dof)
print("Expected counts:")
print(expected)

if p < 0.05:
    print(
        "Reject H0: sex and diagnosis are "
        "associated in this sample."
    )
else:
    print(
        "Fail to reject H0: no significant "
        "association was detected."
    )
```

56. The contingency table contains observed category counts, not averages.
57. The chi-square statistic measures how far the observed counts differ from the counts expected under independence.
58. A statistically significant association does not show that sex causes arrhythmia.

Task: Calculate the arrhythmia percentage for each sex, report chi-square and p, and state the conclusion in plain language.

9.3 Final notebook check for the worked example

59. Restart the notebook kernel and run all cells from the first cell to the last.
60. Confirm that no cell relies on a variable created manually or in a hidden earlier run.
61. Keep all outputs and your own brief explanations beneath each important result or chart.
62. Check that df_original remains raw and df_etl_clean is used for cleaned analysis and testing.
63. Save the completed notebook using a clear filename that includes your name or group number.

Task: Submit and show the complete worked-example notebook to the teaching assistant.

Step 10 — Begin Assessment 1: define the problem and select a dataset

After completing the full worked example, begin the Phase 1 assessment task. Work in your approved group of no more than three students. Start with a meaningful real-world problem, then select a dataset that can support the intended analysis.

64. Define the real-world problem, motivation, stakeholders and practical relevance.
65. State whether the intended task is classification, prediction, clustering, anomaly detection or forecasting.
66. Select an open-source or publicly accessible tabular, alpha-numeric dataset; image-based datasets are not permitted.
67. The dataset must contain more than 20 and fewer than 300 features at the initial preparation stage.
68. It must contain enough records for meaningful exploration, training, validation and testing.
69. It must have suitable documentation, metadata, access terms and a clear source.
70. Check that the target or analytical objective is meaningful and technically feasible.

Task: Prepare a short problem statement and a dataset-suitability table similar to Steps 1 and 2 of this lab.

Step 11 — Seek dataset approval through Canvas

71. Open the Assessment 1 tab on Canvas.
72. Open the online dataset-selection approval form.
73. Enter the group members, proposed project title, problem, dataset name, dataset link and source.
74. Provide the number of records and features, the proposed target or analytical task, and a brief justification.
75. Check that the dataset has not already been selected by another group, where this information is available.
76. Submit the approval form before beginning detailed implementation.

Important: Lecturer approval is required before the group commits to detailed project development (see CANVAS>Assessments>Project Phase 1>Approval form).

Step 12 — Start the Assessment 1 draft

Create a separate notebook or draft document for your own dataset. Apply the principles demonstrated in the worked example, but adapt the code and decisions to the structure and context of your selected dataset.

77. Record the project title, group members, problem statement, motivation, stakeholders and intended analytical objective.
78. Record the dataset name, source, direct link, access date, licence or terms of use, and documentation link.
79. Load the dataset using Python and create an untouched raw copy.
80. Inspect shape, column names, data types, value ranges, unique categories, missing values and duplicate records.
81. Identify potentially invalid or inconsistent values and document the evidence used to interpret them.
82. Propose appropriate cleaning and transformation rules; do not copy Arrhythmia-specific thresholds into an unrelated dataset.
83. Create an initial reusable cleaning function or clearly documented pipeline plan.
84. Produce at least two preliminary visualisations that help explain the dataset or its quality.
85. Write your own brief interpretation of every output included in the draft.

Important: Phase 1 focuses on problem formulation, acquisition, inspection, cleaning and exploratory analysis. Machine-learning modelling is not required in the Phase 1 report.

End-of-lab draft submission

Submit the assessment draft at the end of each lab on CANVAS. The Week 2 draft should be submitted under the **Lab 2 assessment**, and the Week 3 draft should be submitted under the **Lab 3 assessment**.. The draft must include:

86. group-member details and a proposed project title;
87. the meaningful real-world problem and its practical relevance;
88. the proposed dataset, source, documentation, licence and suitability checks;

89. confirmation that the Canvas dataset-approval form has been submitted;
90. initial Python code for loading and inspecting the dataset;
91. initial findings on data types, missingness, duplicates, ranges and unusual values;
92. proposed cleaning and transformation strategies with brief justification;
93. at least two preliminary charts and short interpretations; and
94. a short list of the next actions the group will complete after approval.

Task: Before leaving, confirm that the draft opens correctly, contains the group members' names, and has been submitted successfully.